Lecture 28: Practice Problems Suite (Questions)

Supervised Machine Learning (Part 4) — Decision Trees

This problem set tests algorithmic impurity formulation, continuous threshold search, C4.5 gain ratio regularization, variance reduction, and cost-complexity pruning.

Problem 01: Pure NumPy Shannon Entropy Calculator (ShannonEntropyCalculator)

🎯 Objective & Requirements

Write a production-grade Python/NumPy implementation meeting all mathematical and structural invariants.

Mathematical Invariants:

1. All mathematical formulas must be vectorized in pure NumPy without unvectorized scalar Python loops where possible.
2. Continuous threshold candidate generation must use midpoint interpolation: $\theta_i = \frac{x_i + x_{i+1}}{2}$.
3. Guard against numerical singularities ($0 \log_2 0 \equiv 0$, division by zero with $\epsilon = 10^{-12}$).

Problem 02: Pure NumPy Gini Impurity & Weighted Split Evaluator (GiniImpurityEvaluator)

🎯 Objective & Requirements

Write a production-grade Python/NumPy implementation meeting all mathematical and structural invariants.

Mathematical Invariants:

1. All mathematical formulas must be vectorized in pure NumPy without unvectorized scalar Python loops where possible.
2. Continuous threshold candidate generation must use midpoint interpolation: $\theta_i = \frac{x_i + x_{i+1}}{2}$.
3. Guard against numerical singularities ($0 \log_2 0 \equiv 0$, division by zero with $\epsilon = 10^{-12}$).

Problem 03: Continuous Feature Candidate Threshold & Split Optimizer (ContinuousSplitFinder)

🎯 Objective & Requirements

Write a production-grade Python/NumPy implementation meeting all mathematical and structural invariants.

Mathematical Invariants:

1. All mathematical formulas must be vectorized in pure NumPy without unvectorized scalar Python loops where possible.
2. Continuous threshold candidate generation must use midpoint interpolation: $\theta_i = \frac{x_i + x_{i+1}}{2}$.
3. Guard against numerical singularities ($0 \log_2 0 \equiv 0$, division by zero with $\epsilon = 10^{-12}$).

Problem 04: C4.5 Information Gain Ratio Optimizer (GainRatioOptimizer)

🎯 Objective & Requirements

Write a production-grade Python/NumPy implementation meeting all mathematical and structural invariants.

Mathematical Invariants:

1. All mathematical formulas must be vectorized in pure NumPy without unvectorized scalar Python loops where possible.
2. Continuous threshold candidate generation must use midpoint interpolation: $\theta_i = \frac{x_i + x_{i+1}}{2}$.
3. Guard against numerical singularities ($0 \log_2 0 \equiv 0$, division by zero with $\epsilon = 10^{-12}$).

Problem 05: Regression Tree Variance Reduction Split Evaluator (VarianceReductionSplitter)

🎯 Objective & Requirements

Write a production-grade Python/NumPy implementation meeting all mathematical and structural invariants.

Mathematical Invariants:

1. All mathematical formulas must be vectorized in pure NumPy without unvectorized scalar Python loops where possible.
2. Continuous threshold candidate generation must use midpoint interpolation: $\theta_i = \frac{x_i + x_{i+1}}{2}$.
3. Guard against numerical singularities ($0 \log_2 0 \equiv 0$, division by zero with $\epsilon = 10^{-12}$).

Problem 06: Minimal Cost-Complexity Pruning Path Analyzer (CostComplexityPruningPathAnalyzer)

🎯 Objective & Requirements

Write a production-grade Python/NumPy implementation meeting all mathematical and structural invariants.

Mathematical Invariants:

1. All mathematical formulas must be vectorized in pure NumPy without unvectorized scalar Python loops where possible.
2. Continuous threshold candidate generation must use midpoint interpolation: $\theta_i = \frac{x_i + x_{i+1}}{2}$.
3. Guard against numerical singularities ($0 \log_2 0 \equiv 0$, division by zero with $\epsilon = 10^{-12}$).

Problem 07: Automated Pre-Pruning Cross-Validation Optimizer (PrePruningGridSearchOptimizer)

🎯 Objective & Requirements

Write a production-grade Python/NumPy implementation meeting all mathematical and structural invariants.

Mathematical Invariants:

1. All mathematical formulas must be vectorized in pure NumPy without unvectorized scalar Python loops where possible.
2. Continuous threshold candidate generation must use midpoint interpolation: $\theta_i = \frac{x_i + x_{i+1}}{2}$.
3. Guard against numerical singularities ($0 \log_2 0 \equiv 0$, division by zero with $\epsilon = 10^{-12}$).

Problem 08: ShopSmart E-Commerce Purchase Predictor Pipeline (ShopSmartPurchasePredictorPipeline)

🎯 Objective & Requirements

Write a production-grade Python/NumPy implementation meeting all mathematical and structural invariants.

Mathematical Invariants:

1. All mathematical formulas must be vectorized in pure NumPy without unvectorized scalar Python loops where possible.
2. Continuous threshold candidate generation must use midpoint interpolation: $\theta_i = \frac{x_i + x_{i+1}}{2}$.
3. Guard against numerical singularities ($0 \log_2 0 \equiv 0$, division by zero with $\epsilon = 10^{-12}$).
